ARM Cortex-A. Контролер на прекъсванията (GIC). Вграден логически анализатор (ELA). Монитор на производителността (PMU).

Автор: доц. д-р инж. Любомир Богданов

Съдържание

1. Въведение в GIC
2. Фази на прекъсването
3. GIC и виртуализация
4. Въведение в ELA
5. Инициализация на ELA
6. Програмен модел на ELA
7. Монитор на производителността (PMU)

Въведение в GIC

Generic Interrupt Controller, GIC е модул за прекъсвания, обслужващ ARM Cortex-A ядра.

Еквивалент на NVIC при ARM Cortex-M.

Версия 3 на модула поддържа ядра с микроархитектура: Armv8-A, Armv9-A и Armv8-R.

GIC може да бъде програмиран от процесор през отделен интерфейс.

Въведение в GIC

Групите от регистри, чрез които се програмира модулът, са:

*регистри за **управление** на прекъсванията;

*регистри за **настройка** на поведението (behavior) на прекъсванията;

*регистри за **насочване** (рутиране) на прекъсванията към едно или повече процесорни ядра.

Въведение в GIC

Изключенията (exceptions) имат **нива на привилегия**, които нива указват какви права за достъп има даден софтуер (фърмуер):

***EL0** – нисък приоритет за потребителски приложения (user-space applications);
***EL1** – висок приоритет за ядро на операционна система или друг важен софтуер.

Преминаването от едното ниво в другото става чрез влизането и излизането от изключение.

Въведение в GIC

Прекъсванията могат да бъдат класифицирани в зависимост от приоритетите им в групи:

*__група 0__ (Group 0) – вид изключения с най-висок приоритет;

*__защитена група 1__ (Secure Group 1) – прекъсвания, които трябва да се обслужат на EL1 или EL2 ниво;

*__незащитена група 1__ (Non-Secure Group 1) – прекъсвания, които трябва да се обслужат на незащитено EL1 или EL2 ниво.

Въведение в GIC

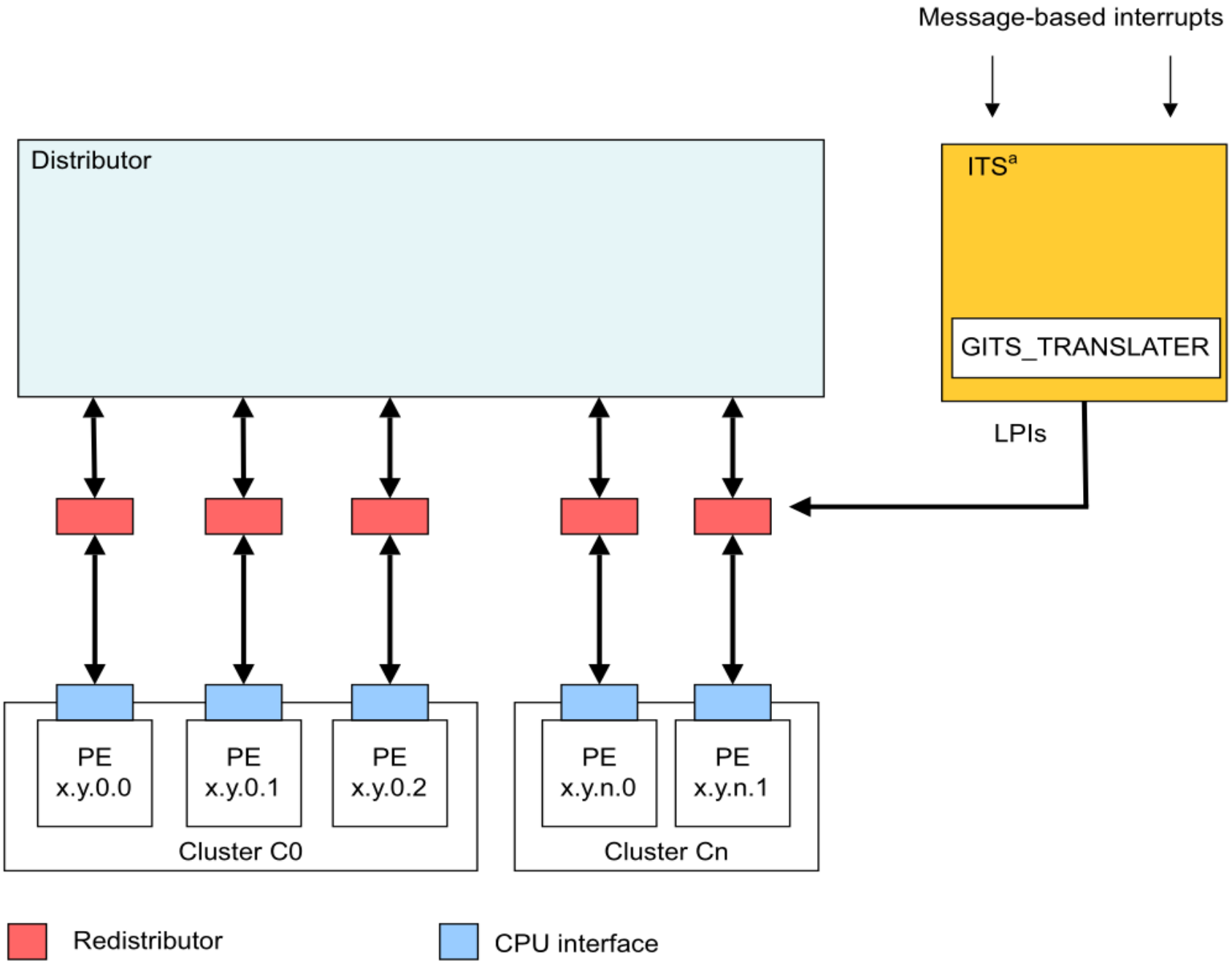
Видовете прекъсвания, поддържани от GIC са:
*__LPI__ (Locally-specific Peripheral Interrupt)
*__PPI__ (Private Peripheral Interrupt)
*__SPI__ (Shared Peripheral Interrupt)
*__SGI__ (Software Generated Interrupt)

Въведение в GIC

***LPI** (Locally-specific Peripheral Interrupt) – това са прекъсвания от незащитена група 1, където има до 2 нива на защита (security). Прекъсванията се активират по фронт.

Това са прекъсвания със съобщения (**message-based interrupts**) – активирането на прекъсването става не чрез един сигнал (проводник), а чрез регистър от адресното поле на системата, в който се записват две числа (DeviceID + EventID).

Въведение в GIC



Въведение в GIC

На миналия слайд:

Процесорен елемент (**PE**, Processing Element) – може да е микропроцесор, хардуерен ускорител, DSP и др.

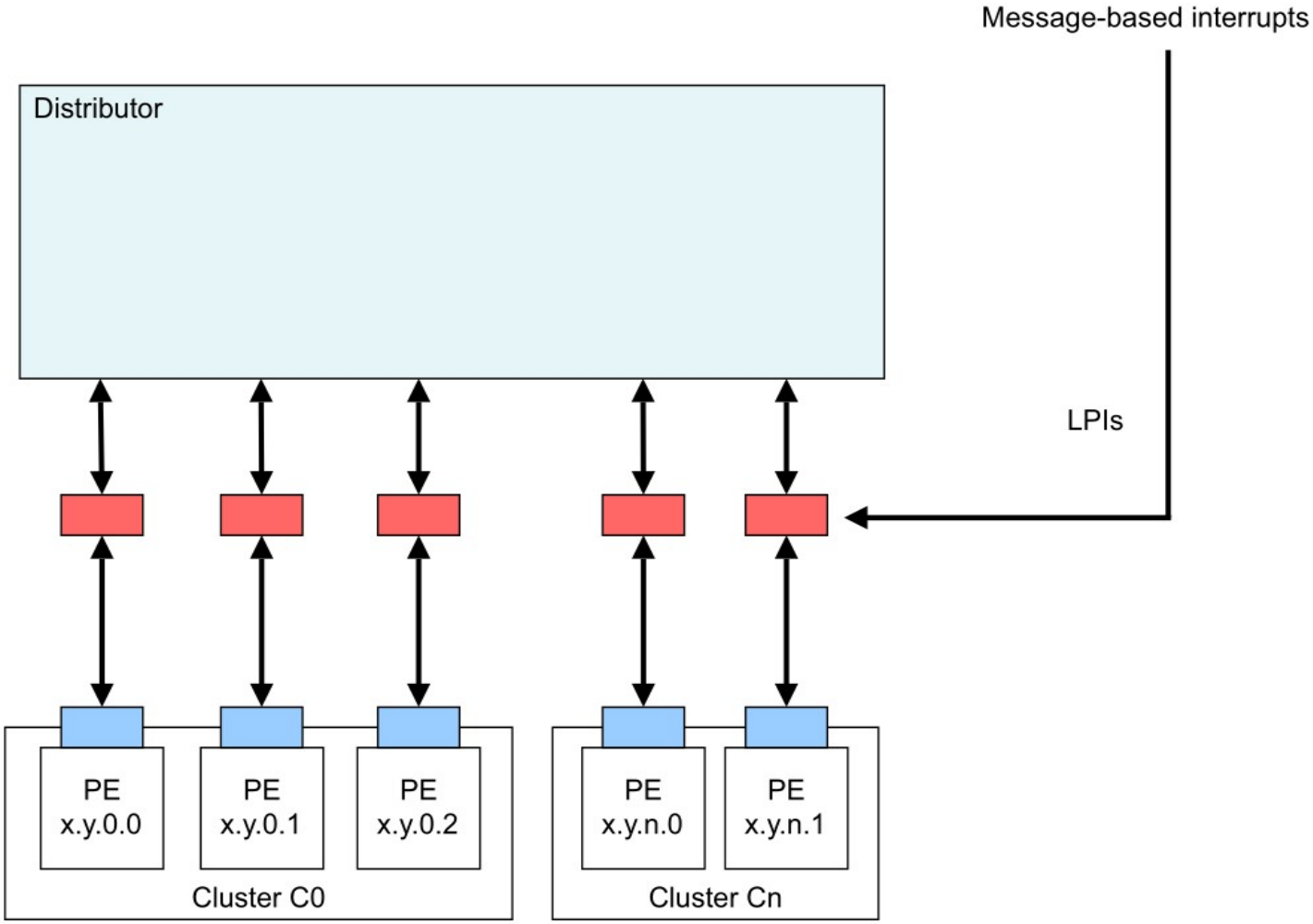
Дистрибутор на прекъсвания (**Distributor**) – разпределя сигнали от прекъсвания към съответния процесорен елемент. Един GIC има един Distributor.

Редистрибутор (**Redistributor**) – локален за процесорното ядро дистрибутор на прекъсвания.

Транслатор на прекъсвания (**ITS**, Interrupt Translation Service) – модул, преобразуващ прекъсване чрез съобщение (**message-based interrupt**) DeviceID + EventID в един електрически сигнал, наречен INTID.

Въведение в GIC

В някои процесорни реализации, ITS блока може да липсва. Тогава LPI прекъсванията се свързват директно към редистрибуторите на прекъсвания.



Въведение в GIC

LPI прекъсванията използват следните регистри:

*__GICR_INVLPIR__ (redistributor invalidate LPI) – изтрива конфигурационните данни за дадено LPI.
*__GICR_INVALLR__ (redistributor invalidate all LPI) – изтрива конфигурационните данни за всички LPI.
*__GICR_SYNCR__ (redistributor synchronize register) – статус регистър, указващ дали всички операции по изтриване за приключили.
*__GICR_SETLPIR__ (set LPI pending register) – записът в този регистър генерира LPI прекъсване.
*__GICR_CLRLPIR__ (clear LPI pending pegister) – записът в този регистър изчиства LPI прекъсване, което предстои да бъде изпълнено (pending).

Въведение в GIC

***PPI** (Private Peripheral Interrupt) – прекъсване, което може да се използва от един-единствен процесорен елемент (PE). Това е прекъсване от група 0, или защитена/незащитена група 1. Прекъсвания от този тип може да се активират по фронт или по ниво. Не минават през ITS.

Пример: прекъсване от Watchdog.

Въведение в GIC

***SPI** (Shared Peripheral Interrupt) – прекъсване, което може да се използва от много процесорни елементи (PE). В даден момент се пренасочва към конретен PE от дистрибутора. Може да е от група 0 или защитена/незащитена група 1. Прекъсвания от този тип може да се активират по фронт или по ниво. Не минават през ITS. Тези прекъсвания може да са едносигнални (wire-based) или съобщителни (message-based).

Пример: прекъсване от UART или GPIO.

Въведение в GIC

SPI прекъсванията използват следните регистри:

GICD_SETSPI_SR – променя статуса на прекъсването от “неактивно” в “предстоящо да бъде изпълнено”. Също променя статуса “активен” в “активен и предстоящо да бъде изпълнено”.

GICD_CLRSPI_SR – променя статуса на прекъсването от “предстоящо да бъде изпълнено” в “неактивно”. Също променя статуса “активен и предстоящо да бъде изпълнено” в “активен”.

GICD_SETSPI_NSR и GICD_CLRSPI_NSR – аналогични на GICD_SETSPI_SR и GICD_CLRSPI_SR, но вземат под внимание статуса на защита (security).

Въведение в GIC

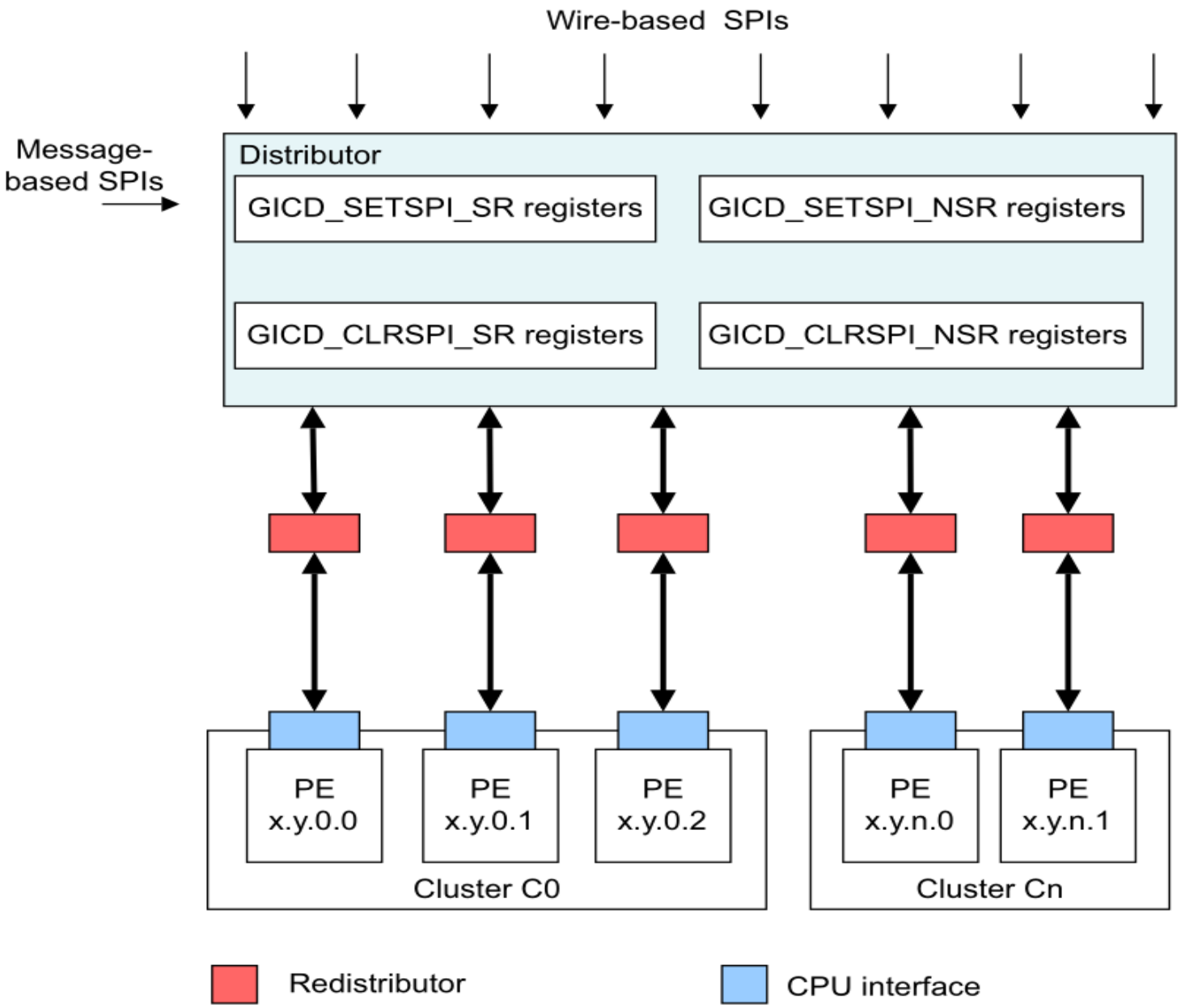


Figure 4-4 Triggering SPIs

Въведение в GIC

***SGI** (Software Generated Interrupt) – прекъсване, което се използва за интер-процесорна комуникация. Активира се чрез запис в регистър от GIC. Може да е от група 0 или защитена/незащитена група 1. Прекъсвания от този тип може да се активират по фронт. Не минават през ITS.

Въведение в GIC

SGI прекъсванията използват следните регистри:

ICC_SGI1R_EL1 – генерира прекъсване от група 1 с ниво на сигурност, което е в момента.

ICC_ASGI1R_EL1 - генерира прекъсване от група 1 с ниво на сигурност различно от това, което е в момента.

ICC_SGI0R_EL1 – генерира прекъсване от група 0, със защита (security).

Въведение в GIC

Видове прекъсвания в зависимост от **бързината на обслужване**:

***FIQ** (Fast Interrupt reQuest) – бързо прекъсване. Използва се за прекъсване с висок приоритет, изискващо бързо обслужване. Това е едно-единствено прекъсване, чийто индекс от векторната талблица е последен [2].

Кодът на хендлера на прекъсването се намира непосредствено след векторната таблица. Това позволява да се избегне инструкция за преход => по-бързо.

Процесорното ядро има **допълнителни регистри** (banked registers), които се използват само от FIQ. Това намалява броя на push и pop в хендлера => по-бързо.

Въведение в GIC

Във FIQ не може да се генерират изключения (като SVC call). Това означава, че не може да се използва от код, който извиква функции на операционната система, понеже тези функции обикновено се извикват чрез SVC.

***IRQ** – стандартни прекъсвания, използващи регистрите от ядрото и имащи индекси във векторната таблица, различни от последния. Техните хендлери се извикват чрез инструкция за преход. Ако се използват повече регистри от регистрите на стековия фрейм (стекова група), трябва да се използват push и pop инструкции.

Фази на прекъсването

Неактивно (**Inactive**) – прекъсването не е активно и не предстои да бъде изпълнено.

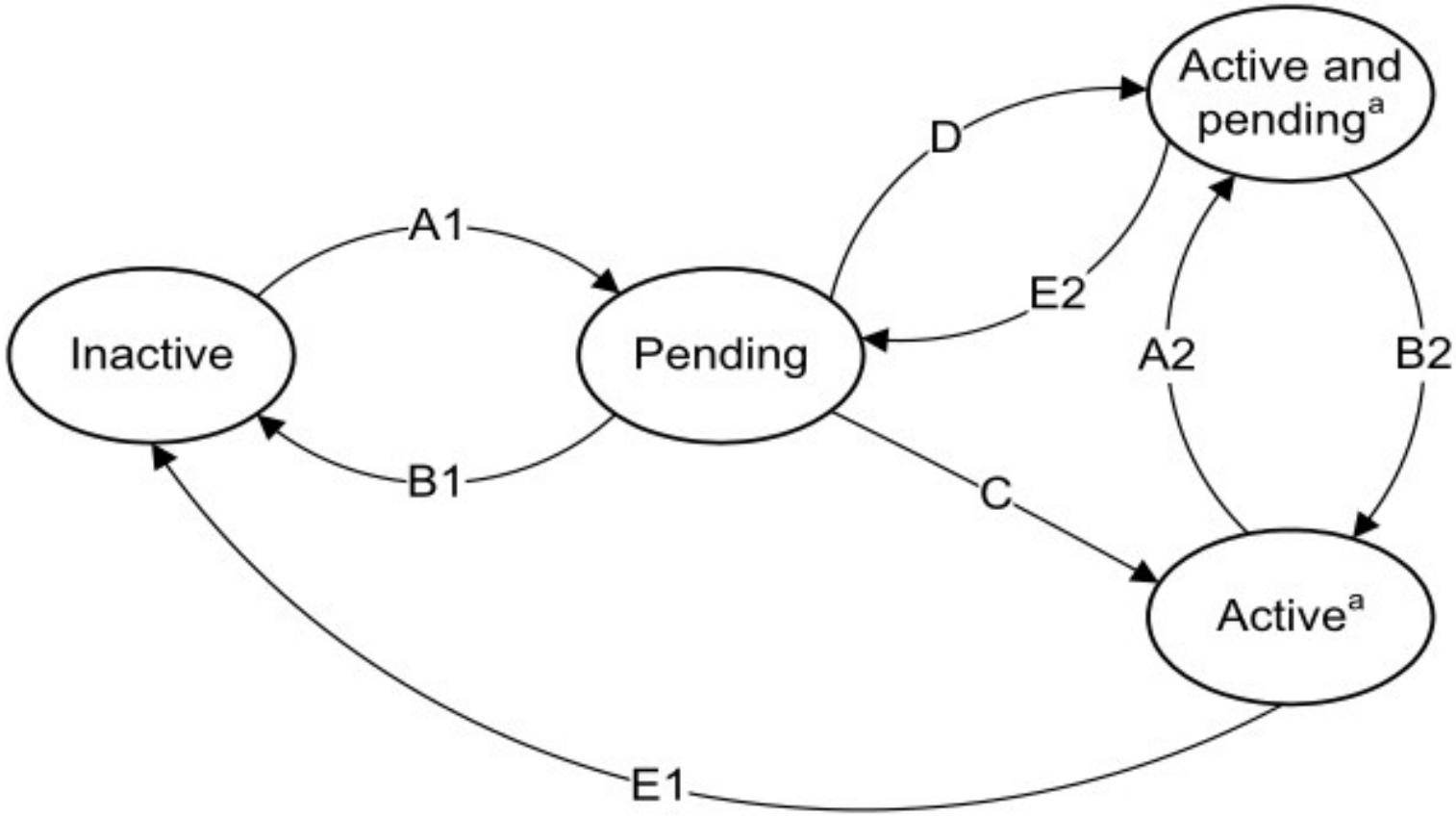
Предстои (**Pending**) – прекъсване, което е активирано от хардуер и предстои да бъде изпълнено, но все още не е.

Активно (**Active**) – прекъсване, което е активирано от хардуер и PE, който ще го изпълнява, е започнал изпълнението на хендлера му. Повторно прекъсване от същия хардуер не е регистрирано и PE изпълнява първото прекъсване.

Активно и престоящо (**Active and pending**) – прекъсване, което е било повторно активирано, докато PE изпълнява хендлера на прекъсването му.

Фази на прекъсването

LPI нямат фаза “Active” и “Active & pending”



a. Not applicable for SGIs when GICD_CTLR.nASSGIreq==1, and for LPIs.

Figure 4-3 Interrupt handling state machine

Фази на прекъсването

Детайлно разглеждане на фазите на едно прекъсване е показано на по-следващия слайд. Фазите са:

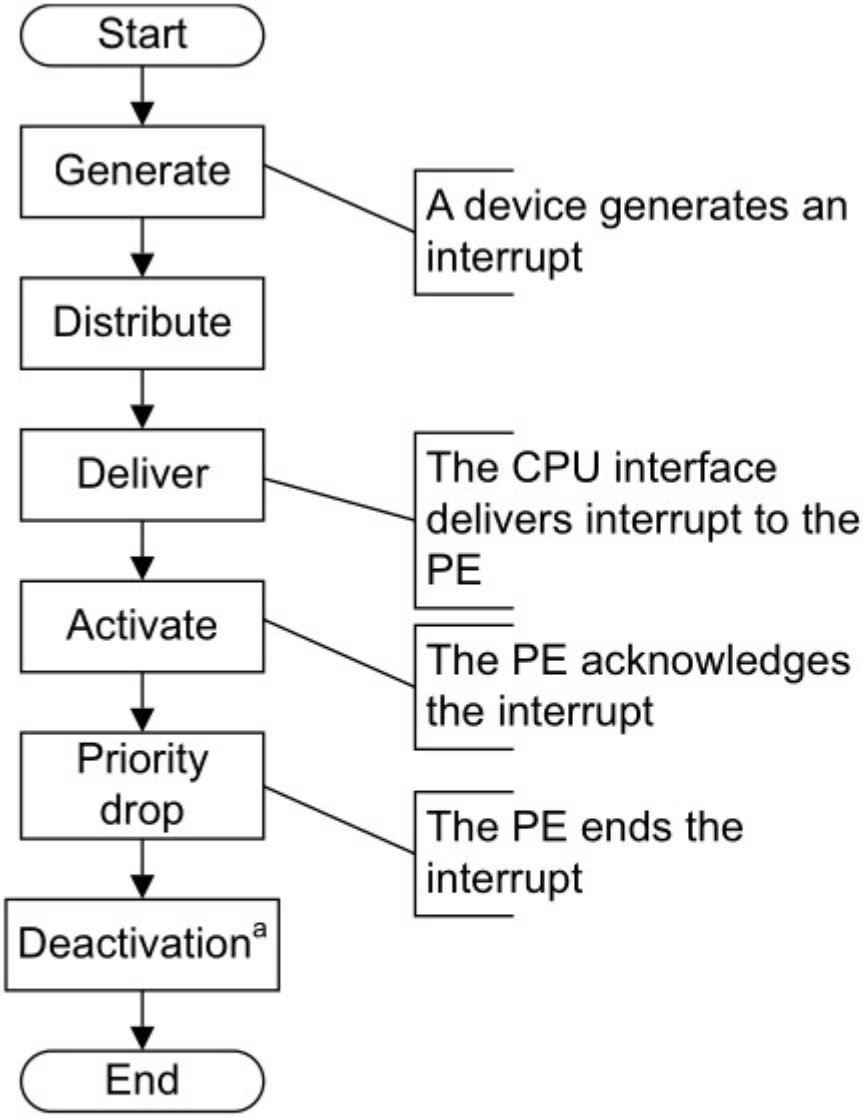
1. Generate – прекъсване е генерирано от периферен хардуерен модул или софтуерно чрез запис в регистър.
2. Distribute – извършва се групиране, приоритизиране и пренасочване на прекъсването до интерфейс на PE.
3. Deliver – интерфейсът на микропроцесора пренася прекъсването до PE.
4. Activate – прекъсването започва да се обслужва. Временно GIC му дава най-висок приоритет (нарича се *running priority*).

Фази на прекъсването

5. Priority drop – временно зададения най-висок приоритет се отнема. Сега прекъсването има приоритетът, който му е бил първоначално зададен. Тази фаза се активира от запис в регистър ICC_EOIxxx (**EOI** – End Of Interrupt).

6. Deactivation – статусът “активен” се премахва. От този момент нататък, същото прекъсване може да бъде активирано отново. Ако това стане, PE ще довърши кода в хенделера на първото прекъсване и ще започне наново процеса на обслужване на второто.

Фази на прекъсването



a. This step does not apply to LPIs.

Figure 4-1 Physical interrupt lifecycle

GIC и виртуализация

Виртуална машина (Virtual Machine, VM) е софтуерна програма, която емулира (имитира) работата на хардуера на един компютър.

В ARM Cortex-A може да се работи с виртуални машини. Тогава операционната система работи на ниво на сигурност EL1 под контрола на хипервайзор (работещ на ниво EL2).

VM може да емулира многопроцесорни системи, което означава, че може да има **повече от един виртуален процесорен елемент** (virtual Processing Elements, vPE).

GIC и виртуализация

Кодът, който ще се изпълнява от vPE, се насочва (диспечерира, **schedule**) към един или няколко реални процесорни елементи PE.

Списък на виртуални прекъсвания – в GIC има набор от регистри, които съдържат информацията за това дали едно прекъсване предстои да се изпълни (pending) или е активно (active) за даден vPE.

В този списък има още информация относно:
*номер на виртуално прекъсване
*група на прекъсване
*статус на прекъсването
*виртуален приоритет

GIC и виртуализация

Прекъсвания от виртуалния списък може да бъдат **обвързани** с физическо (реално) **SPI** или **PPI** прекъсване.

Хипервайзорен системен регистров интерфейс (Hypervisor system register interface) – набор от регистри, чрез които хипервайзорът извършва превключване на контекста (context switch), в случая когато има повече от една виртуална машина. Имената на тези регистри започват с ICH_xxx. Достъпът до тях може да стане само на ниво EL2.

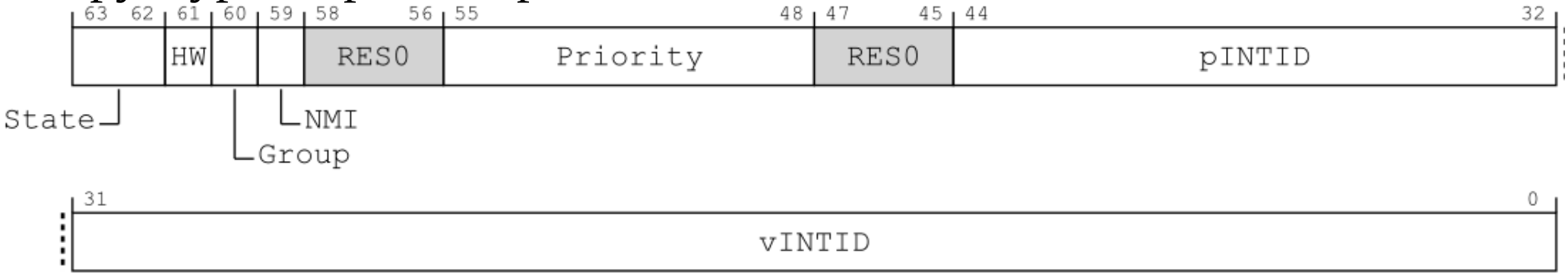
GIC и виртуализация

Регистрите на хипервайзорния регистров интерфейс са:

*ICH_HCR_EL2 – конфигурационен регистър на виртуалните прекъсвания.
*ICH_VTR_EL2 – съдържа информация относно поддържаните от GIC виртуализационни функции.
*ICH_VMCR_EL2 – съдържа емулирани версии на регистри от GIC, които ОС-гост (guest OS) вижда.
*ICH_LR_<n>_EL2 – група от регистри, формираща опашка от прекъсвания на ОС-гост, които трябва да бъдат прехвърлени, през хипервайзора, на реален процесорен елемент PE. (n = 0 – 15)

GIC и виртуализация

Структура на регистър ICH_LR<n>



State – 00 → inactive, 01 → pending, 10 → active, 11 → pending & active.
HW – 0 → прекъсването е генерирано от софтуер и не трябва да се предава към хардуера, 1 → прекъсването е генерирано от виртуален хардуер и трябва да се предаде на реално прекъсване от Дистрибутора.
Group – номер на група 0 или 1 на виртуалното прекъсване.
NMI – флаг за виртуално немаскирано прекъсване.
Priority – приоритет на прекъсването.
pINTID – ID на реалното прекъсване (ако е свързано с виртуалното).
vINTID – ID на виртуалното прекъсване.

GIC и виртуализация

*ICH_ELRSR_EL2 (Empty List Register Status Register) – статус регистър с 15 бита. Всеки бит указва дали регистър ICH_LR_xxx съдържа валидно прекъсване или не.

*ICH_EISR_EL2 (End of Interrupt Status) – статус регистър, съдържащ по един бит за всеки ICH_LR_xxx и отразяващ състоянието EOI (End Of Interrupt) на прекъсването.

GIC и виртуализация

*ICH_MISR_EL2 (Maintenance Interrupt State Register) – контролен регистър, съдържащ битове за глобално включване/изключване на Група 0 & 1 прекъсвания.

*ICH_AP<x>R<n>_EL2 (Active Priority) – x=0-1, n=0-3 – статус регистър, съдържащ 32 бита, отразяващи наличието или неналичието на прекъсване, чийто приоритет все още не е преминал през фазата “priority drop”.

GIC и виртуализация

Когато един реален, **съществуващ хардуерен модул** генерира прекъсване, то може да бъде насочено към виртуална машина, където да се генерира аналогично виртуално прекъсване.

Възможно е операционната система домакин (host OS - тази, която пуска виртуалните машини) да емулира работата на **несъществуващ хардуерен модул**. Тогава също могат да се насочват прекъсвания към дадена виртуална машина.

GIC и виртуализация

ВНИМАНИЕ! Виртуалните и реалните прекъсвания може да имат различни INTID.

Свързване на виртуални прекъсвания с реални (физически) прекъсвания става в няколко последователни стъпки:

1. Хипервайзорът конфигурира регистър ICC_CTLR_EL1 като вдига бит EOImode = 1. Това позволява да се разделят фазите “priority drop” и “deactivate”. Тогава, “priority drop” се инициира от регистър ICC_EOIR<x>_EL1, x = 0-1, а “deactivate” от регистър ICC_DIR_EL1.

GIC и виртуализация

2. PPI или SPI прекъсване се приема от хипервайзора и се потвърждава (acknowledge) от него, което прави прекъсването активно (active).

3. Хипервайзорът попълва една стойност в списъка с виртуални прекъсвания на виртуалния процесор vPE, който трябва да получи прекъсването. Хипервайзорът след това извършва EOI, за да предизвика “priority drop”. Прекъсването, обаче, все още не е във фаза “deactivate”.

GIC и виртуализация

4. Ако виртуалното прекъсване има достатъчно висок приоритет от списъка с прекъсвания за дадения vPE, и кодът на vPE се изпълнява на реален PE, хипервайзорът вдига бит HW от съответния ICH_LR<n> регистър, за да маркира прекъсването като **обвързано**. В същия регистър се записва INTID.

5. Записват се стойности в регистрите:
*първо в ICH_AP0R<n>_EL2,
*след това в ICHAP1R<n>_EL2.
Редът на запис има значение. Ако се обърне редът, конфигурацията ще стане непредсказуема.

Въведение в ELA

Програмна трасираща макроклетка (Program Trace Macrocell) е периферен хардуерен модул от ARM Cortex-A микропроцесора, който позволява да се трасират инструкции в реално време[3].
≤ARMv7

Вградена трасираща клетка (Embedded Trace Macrocell) – е периферен хардуерен модул от ARM Cortex-A микропроцесора, който може да трасира инструкции и данни.
≥ARMv8

Въведение в ELA

```
int main (void)
{
    int i = 10;
    while (i != 0)
     {
        i = sub(i);
     }
}
```

Index	Address	Opcode	Detail
			main+12
-9,999	EL1N:0x0000000080000C70	94000020	BL sub ; 0x80000CF0
			sub
-9,998	EL1N:0x0000000080000CF0	51000400	SUB w0,w0,#1
-9,997	EL1N:0x0000000080000CF4	D65F03C0	RET
			main+16
-9,996	EL1N:0x0000000080000C74	17FFFFFF	B main+12 ; 0x80000C70
-9,995	EL1N:0x0000000080000C70	94000020	BL sub ; 0x80000CF0
			sub
-9,994	EL1N:0x0000000080000CF0	51000400	SUB w0,w0,#1
-9,993	EL1N:0x0000000080000CF4	D65F03C0	RET
			main+16
-9,992	EL1N:0x0000000080000C74	17FFFFFF	B main+12 ; 0x80000C70
-9,991	EL1N:0x0000000080000C70	94000020	BL sub ; 0x80000CF0

Figure 1 Instruction trace

Въведение в ELA

Трасираните инструкции и данни се прехвърлят към дебъгващия компютър посредством high-end дебъгер, използващ високочестотен интерфейс. Информацията се компресира [3]:

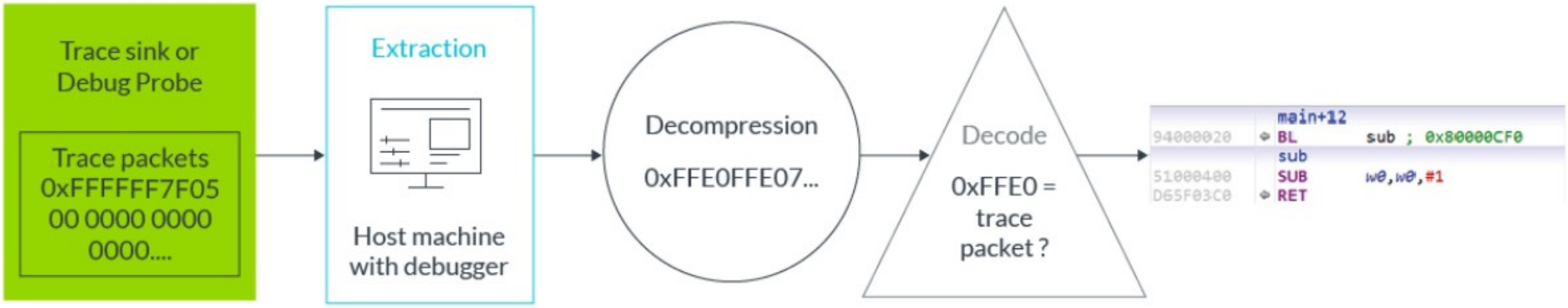


Figure 5 Trace processing stages

Въведение в ELA

Вграден логически анализатор (ELA, Embedded Logic Analyzer) – субмодул от дебъг модула, който се използва за дебъгване на хардуерни проблеми в интегрираните модули на един ARM-базиран микроконтролер.

Сигнали от модула, който се тества, се свързват към ELA и се записва историята на промените им (подобно на осцилоскоп). Казва се, че **сигналите се трасират**. Трасирането става под управлението на тригери. Тригер сигналите може да са единични импулси или поредица от импулси (използват се хардуерни броячи). Данните от ELA модула се прехвърлят към компютър, където се изобразяват в графичен вид.

Въведение в ELA

Към 2025 г. има два варианта на ELA модула [6]:
*ELA-500
*ELA-600

Най-съществените им параметри за показани в таблицата по-долу:

Table 20-2 Comparison of features between ELA-500 and ELA-600

Feature	ELA-500	ELA-600
Trigger states	5	8
Embedded RAM configuration	Yes	Yes
Data compression	No	Yes
ATB interface	No	Yes

Въведение в ELA

Блоква схема на ELA-500 е дадена по-долу [4]:

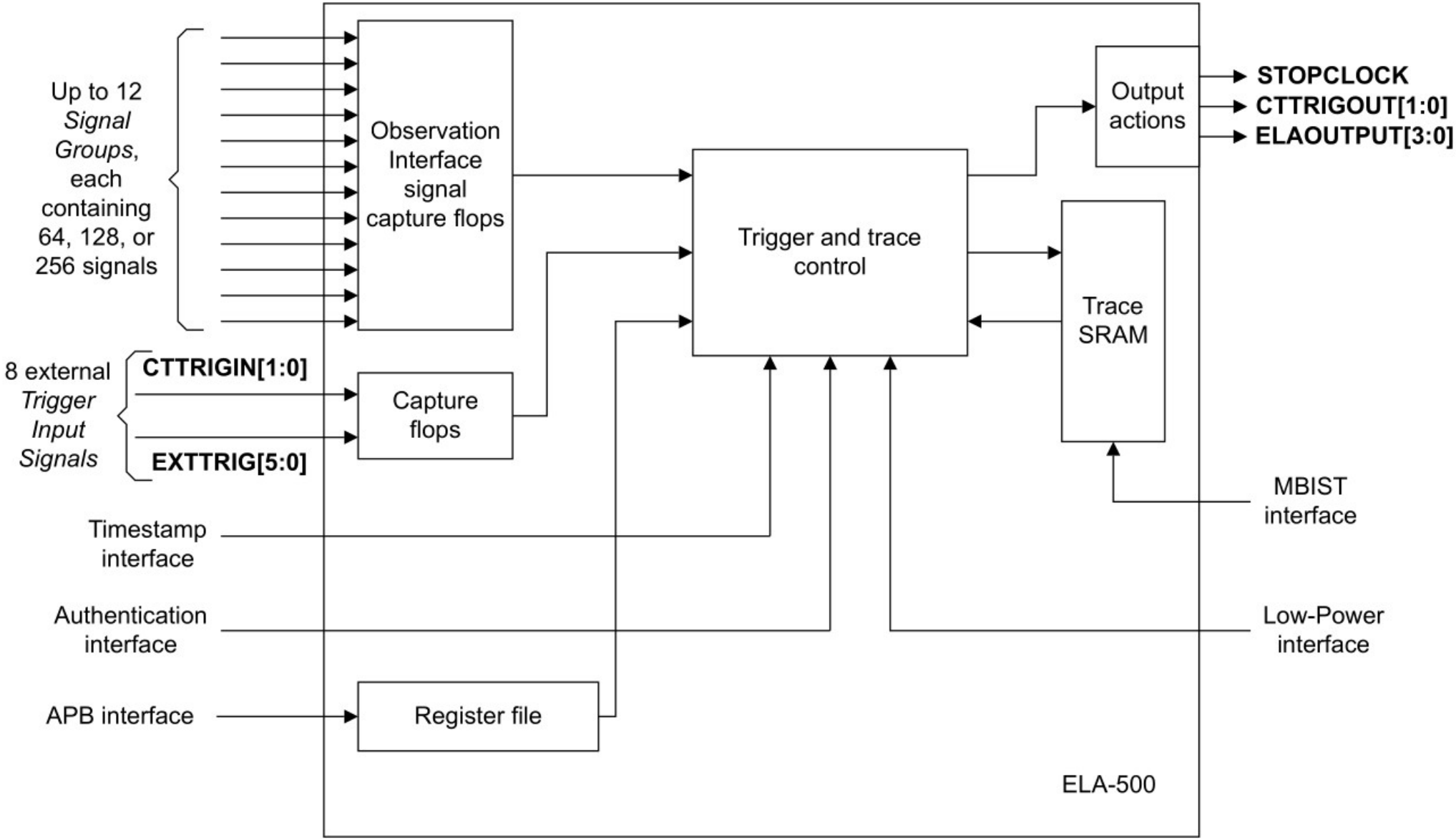


Figure 2-1 ELA-500 block diagram

Въведение в ELA

Блоква схема на ELA-600 е дадена по-долу [5]:

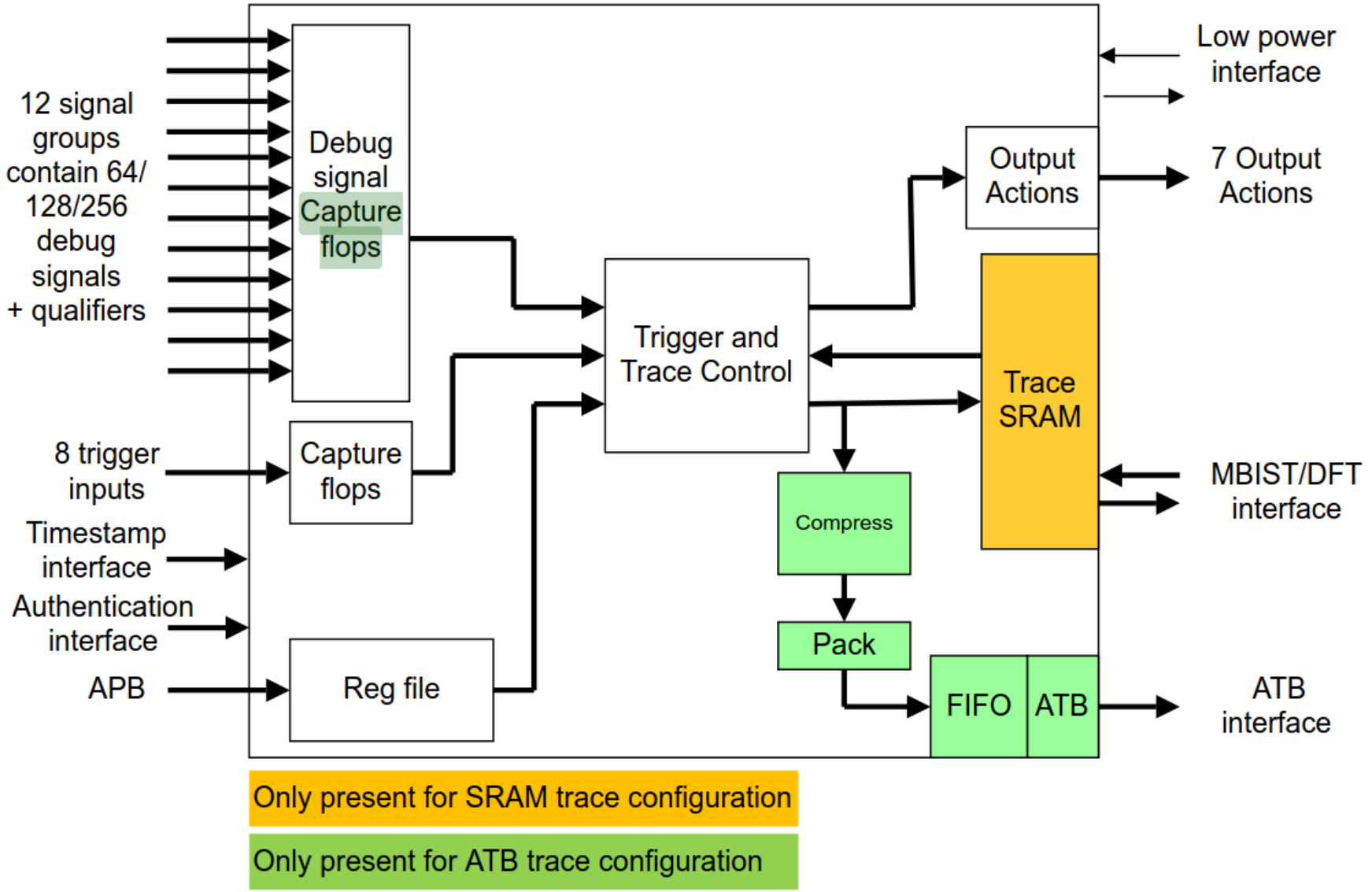


Figure 2-1 ELA-600 processor block diagram

Въведение в ELA

Тригер и контрол на трасирането (Trigger and Trace Control) – блок реализиращ основната функция на ELA модула.

Буферни регистри (capture flops) – сигналите, които ще се трасират първо биват буферирани, за да може записът им да се синхронизира с работата на ELA модула. С помощта на тези буфери сигналите запазват временно състоянието в което са.

Трасираща RAM (Trace RAM) – памет, в която се записват точките от трасираните сигнали.

Въведение в ELA

Изходни действия (Output Actions) – блок за генериране на събития при постъпване на тригер сигнал.

Събитията може да са:

*активиране на STOPCLOCK сигнал, който указва, че вече има данни, които могат да бъдат прочетени.

*активиране на сигнали CTTRIGOUT0 и CTTRIGOUT1, които се използват от CTI (cross trigger interface) CoreSight интерфейс. CTI се използва за синхронизация и реакция на събития от много CoreSight компоненти.

*активиране на сигнали ELAOUTPUT0, ELAOUTPUT1, ELAOUTPUT2, ELAOUTPUT3 с общо предназначение.

Въведение в ELA

Блок за компресия (Compress) – хардуерно компресира данните от трасирането, за да може интерфейсът, по който ще се четат да е по-нискочестотен. Използваната комперсия е: делта компресия.

Блок за форматиране (Pack) – понеже изходните данни от компресията не са кратни на 1байт/1полудума/1 цяла дума, този блок ги форматира да са кратни. Това позволява да бъдат записани в памет на следващ етап.

Блок за буфериране (FIFO) – буферна памет на компресираните и форматирани данни. В случай, че дестинацията не може да поеме повече данни, те се буферират локално в този блок.

Въведение в ELA

Сигнални групи (Signal Groups) – между 64 и 256 сигнала, които ще бъдат трасирани.

Тригерни сигнали (Trigger Input Signals) – до 8 тригериращи сигнали, даващи старт на записването на сигнали от сигналните групи.

Регистров файл (Register file) – група от конфигурационни и статус регистри, с помощта на които се управлява трасирането.

Въведение в ELA

Интерфейс за вътрешна диагностика на RAM памети (MBIST interface, Memory Built-In Self Test interface) – използва се за тест на памети в рамките на един ARM-базиран микроконтролер.

Интерфейс за понижена консумация (Low Power Interface) – свързва се към модул за тактов сигнал. С помощта на този интерфейс ELA може да бъде приведен в режим с понижена консумация. По този интерфейс също се обменя информация дали ELA може да “заспи” или не, ако все още се извършват операции.

Въведение в ELA

Интерфейс за отбелязване на време (Timestamp interface) – по този интерфейс се получава настоящото време от модул за генериране на време. Времето се представя като 64-битова стойност. Това време може да бъде записвано заедно с трасираните данни.

Интерфейс за автентикация (Authentication interface) – конфигурационен интерфейс, с помощта на който се позволява/забранява достъпа до сигнали от ELA. Това е необходимо в случайте, когато има разделяне на защитен-незащитен софтуер и защитеният софтуер трябва да остане скрит (софтуерно и хардуерно чрез сигналите си).

Въведение в ELA

APB интерфейс (Advanced Peripheral Bus) – интерфейс за връзка с микропроцесора ARM Cortex-A. Конфигурационни данни и статус се обменят между ELA модула и ARM Cortex-A посредством APB.

ATB интерфейс (Advanced Trace Bus) – интерфейс за обмен на данни от трасирането между ELA модула и други компоненти от CoreSight дебъг модула.

Въведение в ELA

Запис и четене в трасиращата RAM са показани по-долу:

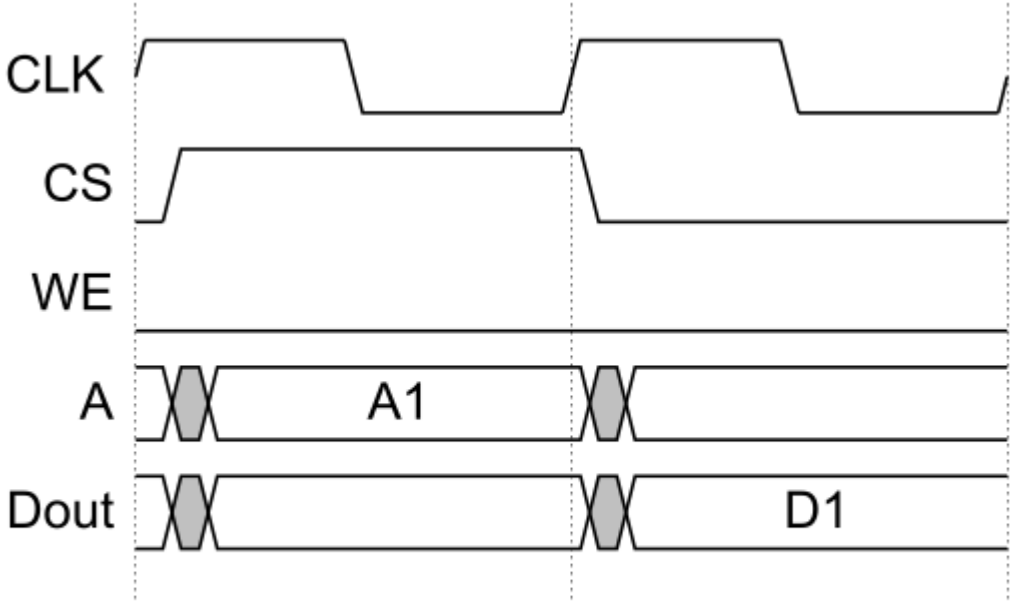


Figure 2-3 SRAM read access timing

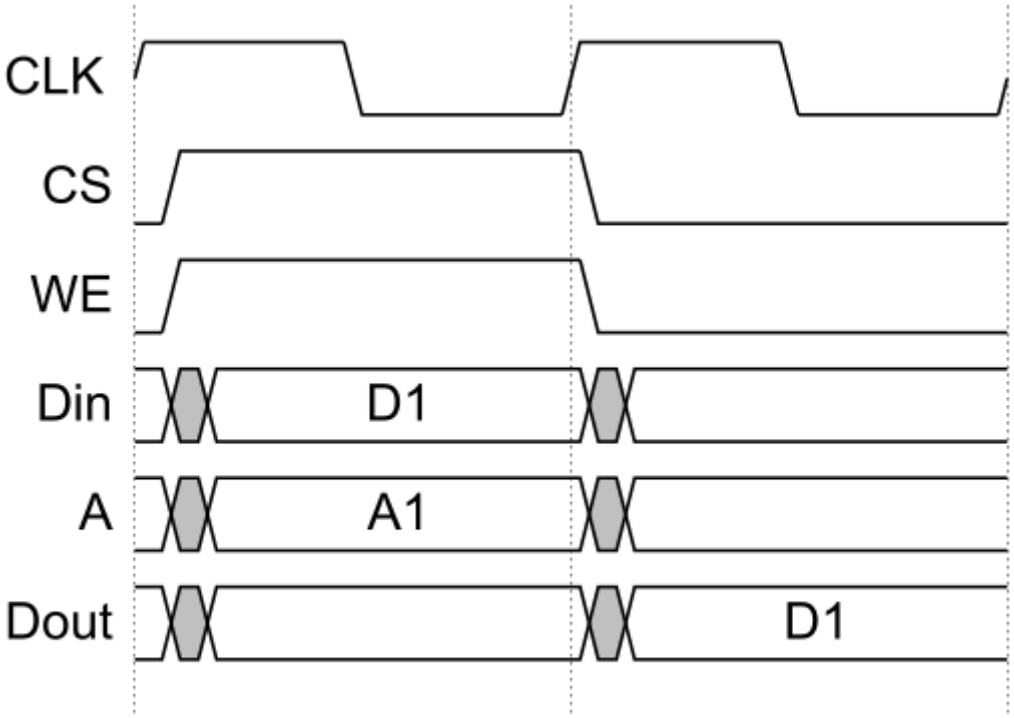


Figure 2-4 SRAM write access timing

Въведение в ELA

Източниците на сигнали по една магистрала са **повече от един** – процесори, DMA, дебъг контролер, периферия от AXI интерфейса и др.

ELA ще запише сигналите от всички източници. За да **може да се различават** един от друг, на всеки източик се дава **ID**.

ID са необходими, понеже **потребителя** обикновено **се интересува само един сигнал** или група от сигнали от една периферия/памет/процесор.

Въведение в ELA

На следващия слайд е показано как два блока “Тригер”, свързани каскадно, различават адрес на достъп и конкретен модул от микроконтролера.

Използват се сигнали “Read address channel” за формиране на ID. Те се свързват от производителя на контролера. Най-често са **няколко бита от AXI** магистралата, която се трасира (“подслушва”).

Въведение в ELA

Формиране на ID на сигнала [4], [5].

Даннова магистрала = read data channel,

адресна магистрала = address value,

контролна магистрала = read address channel.

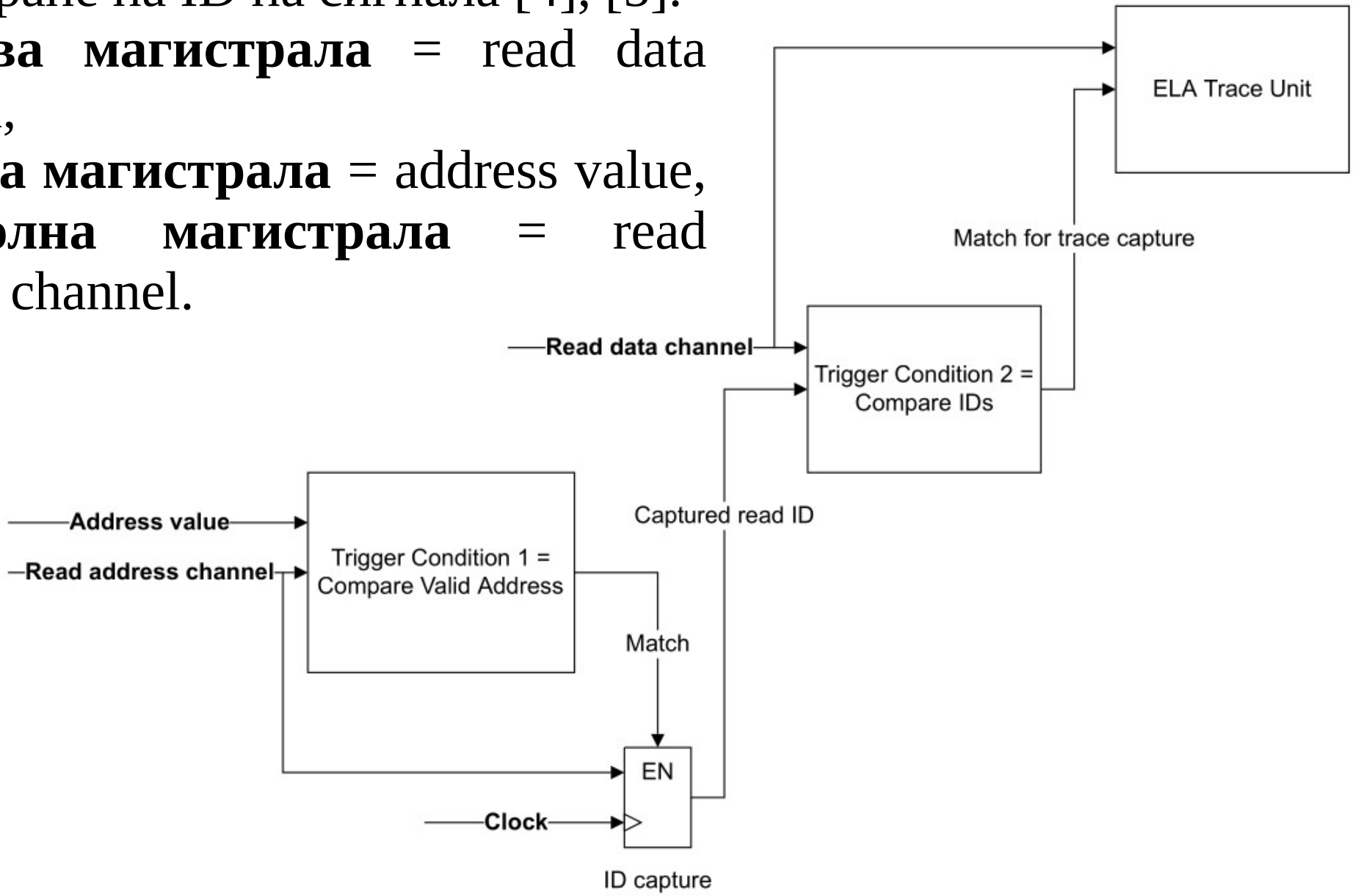


Figure 2-8 Transaction ID capture from read address channel

Инициализация на ELA

Инициализацита на ELA се извършва в няколко стъпки [5]:

1. Спира се ELA чрез зануляване на бит RUN от **CTRL** регистъра. Това позволява следващи записи в конфигуриращите регистри на модула.
2. Записват се стойности в **SIGSEL<n>** регистрите, с което се избират дебъг магистралите в групи Signal Group 0 – 11.
3. Записват се стойности в регистри **SIGMASK<n>** и **SIGCOMP<n>** за съответно маските и събитията за сравнение, при които ще настъпи тригериране.
4. Конфигурира се вида на изходното събитие, което ще се генерира при сравнение, чрез **ACTION<n>** регистрите.

Инициализация на ELA

5. Записват се стойности в **NEXTSTATE<n>** регистрите, които указват вторични тригер събития. След ресет се използват първичните тригер събития.

6. Ако се използват броячи на събития, трябва да се запише стойност на съвпадение (compare value) в регистър **COUNTCOMP<n>** за всяко едно тригер събитие.

7. Записват се стойности в **TRIGCTRL<n>** регистрите за настройка на броячите и вида на compare събитието (=, <, >=, !=, <, <=).

8. Записва се стойност в **RWAR** регистъра, който указва начален адрес от трасиращата RAM. Поне един байт трябва да бъде трасиран.

9. Бит RUN от **CTRL** регистъра се установява в 1. Модулът ELA започва да трасира сигнали.

Програмен модел на ELA

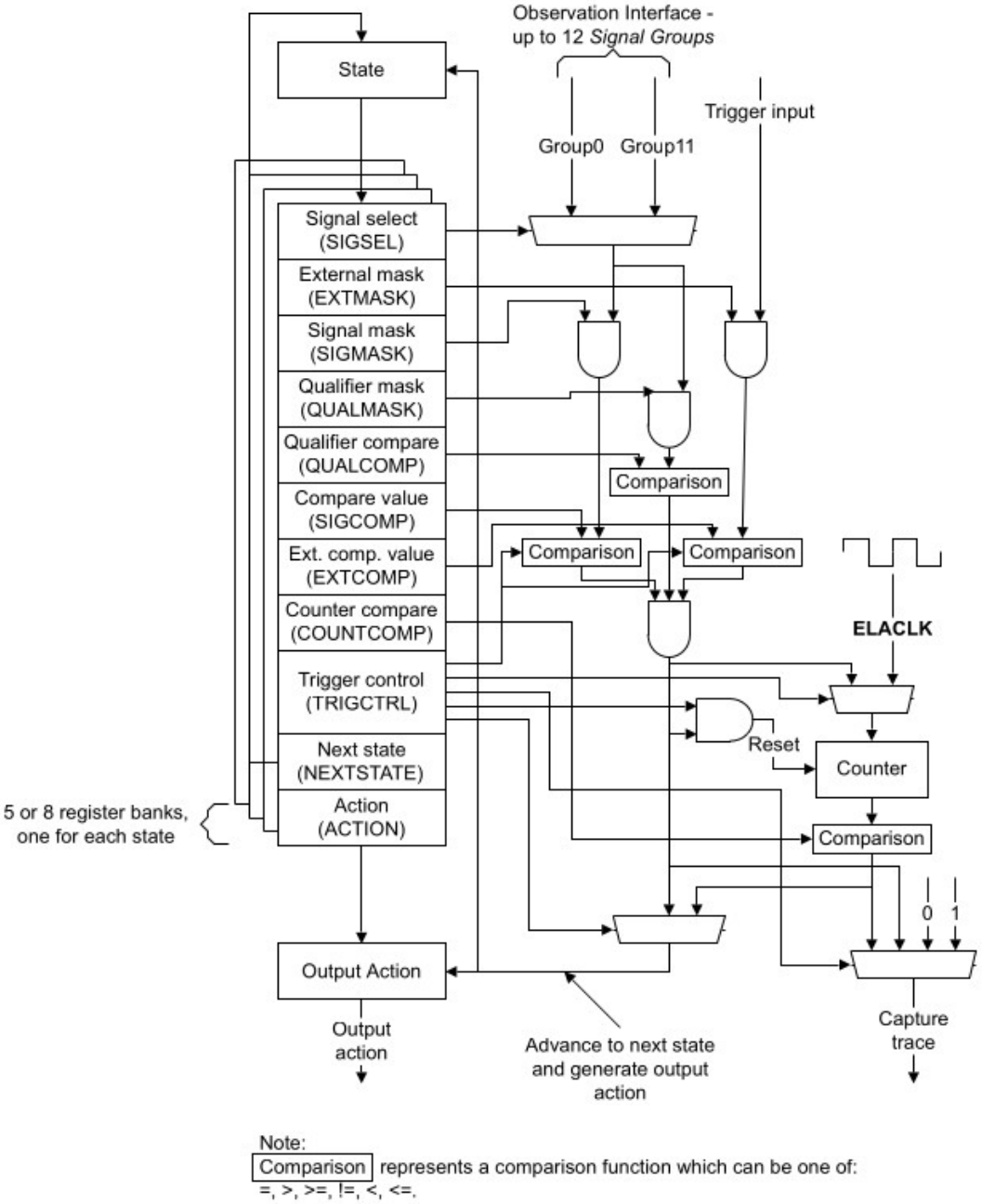


Figure 2-7 ELA-600 triggering mechanism

Програмният модел е показан чрез блоковата схема вляво. Детайлно описание на регистрите има в [5] и [6].

Важно: observation interface е аналогичен на “сонди на логически анализатор”. Източниците на сигнали обикновено са фискирани при производството на микроконтролера (от производителя например NXP, ST, Ti).

Програмен модел на ELA

Пример за трасиране на сигнал е даден на следващия слайд. Данните са взети от [6].

Използва се средата ARM Development Studio.

Пайтън скриптове контролират процеса:

ela_control.py – старт, стоп, статус принт, четене на буфер.

ela_example.py – пример с периодично трасиране.

ela_lowlevel.py – конфигурация на тригер контролни регистри.

ela_test.py – генериране на произволни данни, които да запълнят ELA буфера.

process_trace.py – декомпресия и декодиране.

ela_setup.py – конфигурация на тригер контролни регистри + стартиране на периодично трасиране.

Резултатите се представят в текстови вид (засега).

Програмен модел на ELA

f. Right-click **Decompress and decode ELA trace**, and select **Run ela_process_trace.py::Decompress and decode ELA trace**.

Decoding the data, based on the configured signal groups, turns it into something like this:

```
Trace type: Data, Trace Stream: 0, Overrun: 0, Data:
0x80300162000003481C00400028082D07
P1_VALID : 1'h1
P1_AXID : 12'h6
P1_addr : 42'hB1000000
P1non-secure : 1'h0 => secure
Type_P1 : 4'hD => Exclusive Read
P0_VALID : 1'h0
P0_AXID : 12'h40E
P0_addr : 42'h80005010
P0non-secure : 1'h0 => secure
Type_P0 : 4'h2 => Read Shared, Read Clean, Read No Snoop Dirty
TTID_P1 : 6'h34
TTID_P0 : 6'h7
Trace type: Data, Trace Stream: 0, Overrun: 0, Data:
0xA0300162000002C81C00400000602675
P1_VALID : 1'h1
P1_AXID : 12'h406
P1_addr : 42'hB1000000
P1non-secure : 1'h0 => secure
Type_P1 : 4'hB => Write Back, Writes Clean
P0_VALID : 1'h0
P0_AXID : 12'h40E
P0_addr : 42'h800000C0
P0non-secure : 1'h0 => secure
Type_P0 : 4'h2 => Read Shared, Read Clean, Read No Snoop Dirty
TTID_P1 : 6'h19
TTID_P0 : 6'h35
Trace type: Data, Trace Stream: 0, Overrun: 0, Data:
0x80400162000006506C005881F7C02C74
P1_VALID : 1'h1
P1_AXID : 12'h8
P1_addr : 42'hB1000000
P1non-secure : 1'h1 => non-secure
Type_P1 : 4'h9 => Write No Snoop
P0_VALID : 1'h0
P0_AXID : 12'h836
P0_addr : 42'hB103EF80
P0non-secure : 1'h0 => secure
Type_P0 : 4'h2 => Read Shared, Read Clean, Read No Snoop Dirty
TTID_P1 : 6'h31
TTID_P0 : 6'h34
```

Монитор на производителността

Монитор на производителността (PMU, Performance Monitor Unit) – е модул на ARM Cortex-A микропроцесорите, който събира статистически данни относно производителността на ядрото и системната памет по време на изпълнение на код.

Използва се за **профилиране на код** (code profiling) – това означава да се оцени бързодействието му.

Монитор на производителността

PMU съдържа **6 брояча**.

Всеки брояч може да **брои събития**, поддържани от PMU за дадения микропроцесор.

Блокова схема на PMU от ARM Cortex A32 микропроцесор е дадена на следващия слайд [7].

Монитор на производителността

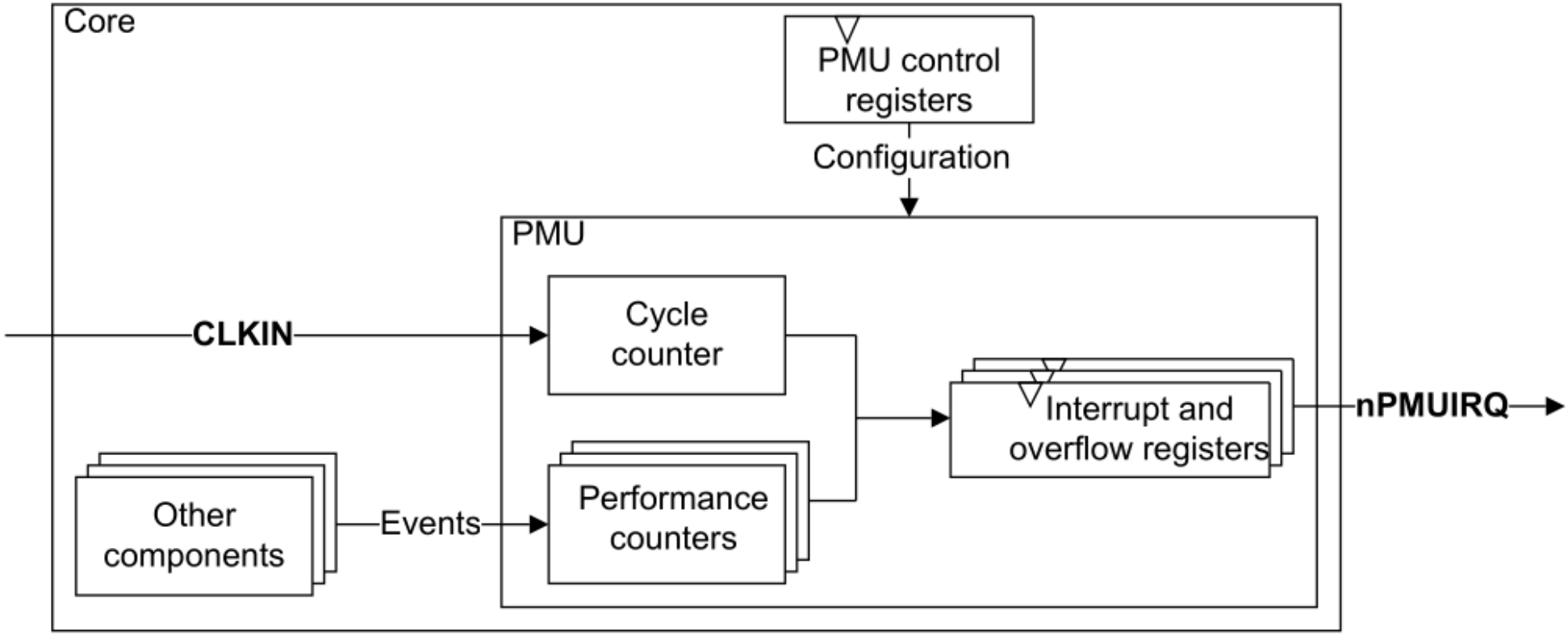


Figure C2-1 PMU block diagram

Монитор на производителността

CLKIN – вход за тактов сигнал. Свързва се към тактовия сигнал на ядрото.

nPMUIRQ – сигнал за прекъсване от PMU. Свързва се към GIC и кол CTI интерфейса на ELA модул (ако има).

Cycle count – 64-битов брояч на тактови импулси. Използва се за времева база на останалите измервания.

PMU control registers – контролни регистри на блока.

Монитор на производителността

Performance counters – 6 брояча на събития.

Interrupt and overflow registers – регистри за генериране на прекъсване при съвпадение.

Other components – други компоненти.

Монитор на производителността

Видовете събития, които се броят са:

* L1 кеш пропуск на инструкция
* L1 кеш зареждане (refill) на TLB от L2 кеш
* L1 кеш невалидиране
* изключение, което е започнало изпълнение
* изключение, което е приключило (return)
* брой процесорни цикли
* брой процесорни цикли при блокаж (stall)
* предсказан преход, който е бил взет
* предсказан преход, който не е бил взет
* ECC грешки в RAM
* достъп до RAM за запис/четене
* спекулативно изпълнена инструкция за промяна на PC

и много други ...

Монитор на производителността

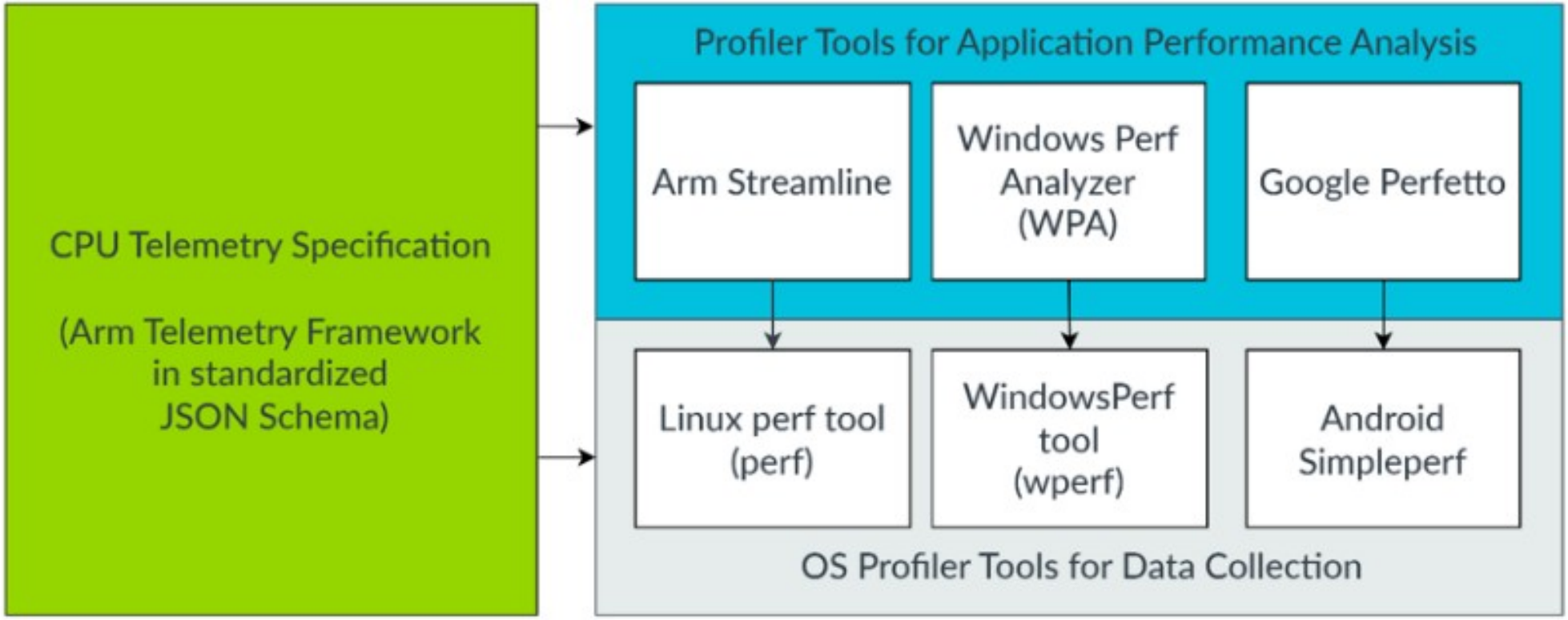
PMU позволява да се чете статистиката за CPU ядрото от следните програми [9]:

*ARM Streamline

*Perf

*Google Perfetto

Figure 4-1: Arm Telemetry Solution and profiling tool enablement



Монитор на производителността

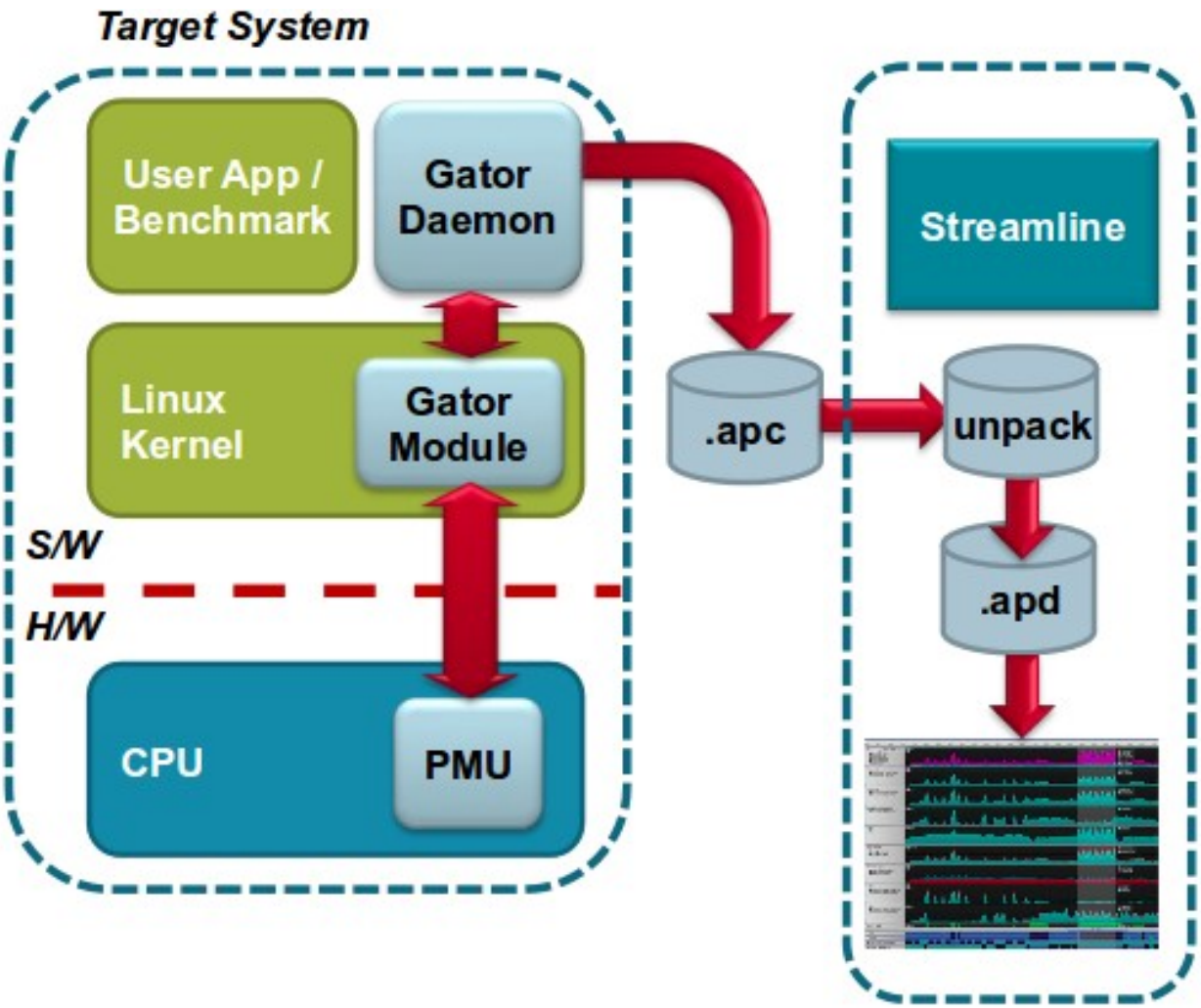
Четенето на PMU става от драйвер за Линукс (ако системата има Линукс), наречен **Gator**.

Сървърното приложение **Gatord** се свързва с драйвера и компресира данните от броячите на производителност.

Gatord може да се свърже например с ARM Development Studio (преди ARM DS-5) и да изпраща данните чрез **TCP/IP протокол** по Етернет интерфейс (ако системата има).

ARM Development Studio изобразява данните в **графичен вид** като осцилограми.

Монитор на производителността



- Relies on “gator” kernel module and daemon
- Reads out counters and process information and dumps to file

Монитор на производителността

C/C++ - /home/lbogdanov/Desktop/Capture_04.apc - Eclipse

File Edit Source Refactor Navigate Search Project Run Window Help

Quick Access C/C++

Project Ex

10x20_LED_master_i2
10x20_LED_matrix_te
11192014_msp430g2
11192014_msp430g2
AM335X_StarterWare
blinky_msp430g2553
blinky_msp430g2553
i2c_master
i2c_slave
msp430g2553_4asovr
MSP430G2553_uartst
seconddrv_2

Outline

An outline is not available.

Streamline Data

192.168.7.2:3456

Filter

Capture_03

Capture_04

Capture_03 Capture_04

Timeline Functions

50ms

0s 0.5s 1s 1.5s 2s 2.5s 3s 3.5s 4s 4.5s 5s 5.5s 6s 6.5s 7s 7.5s 8s 8.5s 9s 9.5s 10

CPU Activity
User
System
100%

Branch
Mispredicted
400 K

Clock
1.1 GHz

[for_loops.bin #2...

Heat Map

Filter map processes

Problems Tasks Console Properties Call Graph

Click here to hide all windows and show the desktop.

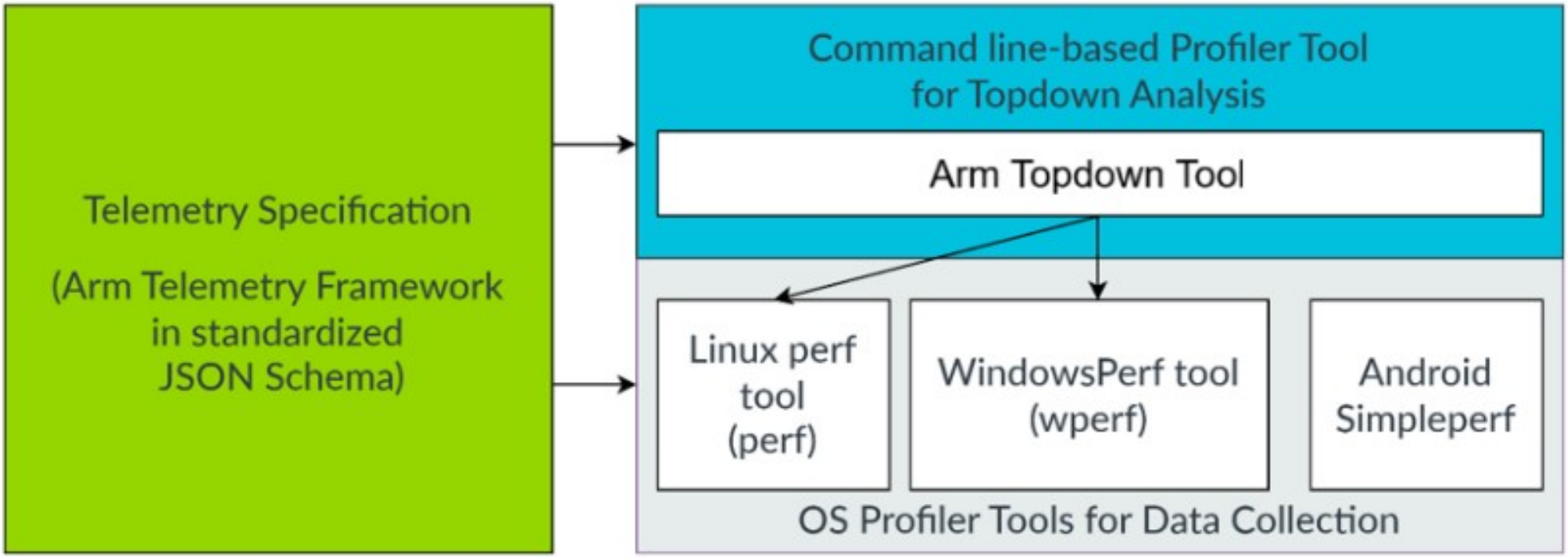
Open source ARM Streamline Performance Analyzer plug-in + open source Eclipse

Монитор на производителността

Четенето на PMU може да стане на самата система, която се тества (ако тя има Linux или Windows).

Тогава може да се използва програмата **ARM Topdown** [9].

Figure 4-2: Arm Topdown tool



Монитор на производителността

JSON Schema – описание на събитията, които могат да бъдат мониторирани от дадения процесор в JSON формат.

Монитор на производителността

Topdown използва пакетът от програми **perf** (Линукс, за Андроид Линукс **- simpleperf**) и **wperf** (Уиндоус).

Използват се програмите:

*__perf-stat__ – оценка на бързодействието на изпълнявания код

*__perf-record__ – записване на бързодействието на всички потребителски и библиотечни функции на равни интервали от време

*__perf-report__ – генериране на обобщена извадка на данните, записани с perf-record

*__perf-anotate__ – генериране на дисасемблерен листинг със записани процентите на натоварване в коментарите.

*__perf-stat__ – 10 секунди брои всички инструкции на всички процесорни ядра.

Монитор на производителността

За да е възможна работата на perf, Линукс ядрото трябва да е компилирано с активиран CONFIG_HW_PERF_EVENTS макрос в конфигурационния му файл.

Налага се изпълнението на две команди при стартиране на OS (само за тестване, защото е security breach):

```
echo -1 > /proc/sys/kernel/perf_event_paranoid *
echo 0 > /proc/sys/kernel/kptr_restrict **
```

*Разрешаване на събития, които могат да разкрият тайна информация

**Разрешаване на разкриване на адресите на ядрото

Монитор на производителността

Примери:

```
topdown-tool --cpu-stages topdown ./a.out
```

Results for Topdown level 1:

```
CPU Neoverse V2 metrics
└── Stage 1 (Topdown metrics)
    └── Topdown Level 1 (Topdown_L1)
        └──
            ____________________________________
            | Metric           | Value | Unit |
            |__________________|_______|______|
            | Backend Bound    | 98.00 | %    |
            | Bad Speculation  | 0.01  | %    |
            | Frontend Bound   | 0.05  | %    |
            | Retiring         | 1.82  | %    |
            |__________________________________|
```

Based on these results, this workload shows a very high backend bound metric.

Монитор на производителността

Примери:

Step 2, Collect the stage 2 Microarchitecture Analysis metrics

In this step, we can specify the metric groups of interest for further analysis. In the following example, the cache effectiveness is checked for a high backend bound workload.

```
topdown-tool --cpu-metric-group L1D_Cache_Effectiveness,L2_Cache_Effectiveness ./
a.out
```

Results for Stage 2:

```
CPU Neoverse V2 metrics
└── Stage 2 (uarch metrics)
    ├── L1 Data Cache Effectiveness (L1D_Cache_Effectiveness)
    │   ├── Follows
    │   │   └── Backend Bound (backend_bound)
    │   └── _______________________________________________________
    │       | Metric                | Value   | Unit                |
    │       |_______________________|_________|_____________________|
    │       | L1D Cache Miss Ratio  | 0.972   | per cache access    |
    │       | L1D Cache MPKI        | 476.998 | misses per 1,000 instructions |
    │       |                       |         |                     |
    └── L2 Unified Cache Effectiveness (L2_Cache_Effectiveness)
        ├── Follows
        │   ├── Backend Bound (backend_bound)
        │   └── Frontend Bound (frontend_bound)
        └── _______________________________________________________
            | Metric                | Value   | Unit                |
            |_______________________|_________|_____________________|
            | L2 Cache Miss Ratio   | 0.014   | per cache access    |
            | L2 Cache MPKI         | 13.093  | misses per 1,000 instructions |
            |_______________________|_________|_____________________|
```


Based on these results, there are very high L1D cache misses.

Монитор на производителността

Примери:

There are many metric groups to explore. For example, we can check the proportions of different kinds of instructions, for example, the Operation Mix metric.

```
topdown-tool --cpu-metric-group Operation_Mix ./a.out
```

Results for Speculative Operation Mix:

```
CPU Neoverse V2 metrics
└── Stage 2 (uarch metrics)
    └── Speculative Operation Mix (Operation_Mix)
        ├── Follows
        │   ├── Backend Bound (backend_bound)
        │   └── Retiring (retiring)
        └── 
            _____________________________________________________________
            | Metric                                          | Value | Unit |
            |_________________________________________________|_______|______|
            | Barrier Operations Percentage                   | 0.02  | %    |
            | Branch Operations Percentage                    | 48.10 | %    |
            | Crypto Operations Percentage                    | 0.00  | %    |
            | Integer Operations Percentage                   | 2.85  | %    |
            | Load Operations Percentage                      | 47.61 | %    |
            | Floating Point Operations Percentage            | 0.00  | %    |
            | Advanced SIMD Operations Percentage             | 0.00  | %    |
            | Store Operations Percentage                     | 0.79  | %    |
            | SVE Operations (Load/Store Inclusive) Percentage | 0.00  | %    |
            |_________________________________________________|_______|______|
```

Литература

[1] “ARM Generic Interrupt Controller Architecture Specification”, ARM IHI0069H.b (ID041224), ARM Ltd, ver. 3 & ver. 4, 2024.

[2] “ARM Cortex-A Series (Armv7-A) Programmer’s Guide”, ver.4.0, denn0013_0400_en, ARM Ltd, 2012.

[3] “Understanding Trace”, 102119_0200_00, version 2.0, ARM Ltd, 2019.

[4] “ARM Coresight ELA-500 Embedded Logic Analyzer”, Technical Reference Manual, Rev. r2p2, ARM Ltd, 100127_0202_01_en, 2018.

[5] “ARM Coresight ELA-600 Embedded Logic Analyzer”, Technical Reference Manual, Rev. r2p0, ARM Ltd, 101088_0200_05_en, 2020.

[6] “ARM Development Studio”, User Guide, ver. 2020.1, ARM Ltd, 101470_2010_00, 2020.

[7] “ARM Cortex-A32 Processor”, Technical Reference Manual, rev. r1p0, 100241_0100_00_en, ARM Ltd, 2019.

[8] “ARM Cortex-A78 Core”, Technical Reference Manual, rev. r1p2, 101430_0102_09_en, Issue 09, ARM Ltd, 2023.

[9] “ARM CPU Telemetry Solution”, Topdown Methodology Specification, 109542_02_02_en, Issue 02, ARM Ltd, 2025.